Heart Disease Risk Prediction: an End-to-End MLOps Pipeline

Machine Learning Operations (AIMLCZG523), Assignment 1
Name: Amar Kumar Biswal | **BITS ID:** 2025AE05569
Code repository: https://github.com/amarbiswalbitspilani/heart-disease-mlops | Video walkthrough: https://drive.google.com/file/d/1YUYkeGyjGZf7dX4_nhorlyornJSBLwSc/view?usp=sharing

1. Problem, data and setup

Goal. Predict whether a patient has heart disease from routine clinical measurements, and serve that prediction as a monitored API. The input is one patient record (13 attributes); the output is a class (disease / no disease) and a confidence. A missed disease case (false negative) is the costly error, the course's *Type 2 error*; the rule for that case is "reduce false -ve (type 2 error)" and "recall (optimize your model)" (CS-4, p17). Recall is therefore the priority metric, and the model is selected on F2, which weights recall twice as much as precision.

Dataset. Heart Disease, UCI Machine Learning Repository (dataset id 45), processed Cleveland file: 303 patients and 14 attributes, i.e. 13 clinical features plus `num`, a disease grade from 0 to 4; these are the 14 attributes the brief refers to. The binary target is `num > 0` (disease present).

Setup and install (Python 3.11 through `uv`, no administrator rights needed; full commands in the repository README). The same steps run on a clean machine:

```
git clone https://github.com/amarbiswalbitspilani/heart-disease-mlops.git && cd heart-disease-mlops
curl -LsSf https://astral.sh/uv/install.sh | sh && export PATH="$HOME/.local/bin:$PATH"
uv venv --python 3.11 --seed .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py                # UCI download, cleaned CSV
PYTHONPATH=src python -m heart.train           # tune, track in MLflow, register, save the model
python scripts/predict.py                      # score api/sample_input.json
python -m pytest && ruff check .               # unit tests and lint
docker build -t heart-api:1.0.0 . && docker run -d --name heart-api -p 8000:8000 heart-api:1.0.0
```

Access for local testing. The container serves `http://localhost:8000` (`/predict`, `/health`, `/metrics`, and interactive docs at `/docs`). On Minikube the API is reached through the LoadBalancer's EXTERNAL-IP (`kubectl -n heart get svc heart-api`); in this deployment it was `http://192.168.49.200`. Versions used: Python 3.11.16, Docker 29.9.1, Minikube v1.39.0, Kubernetes v1.37.0.

2. Architecture

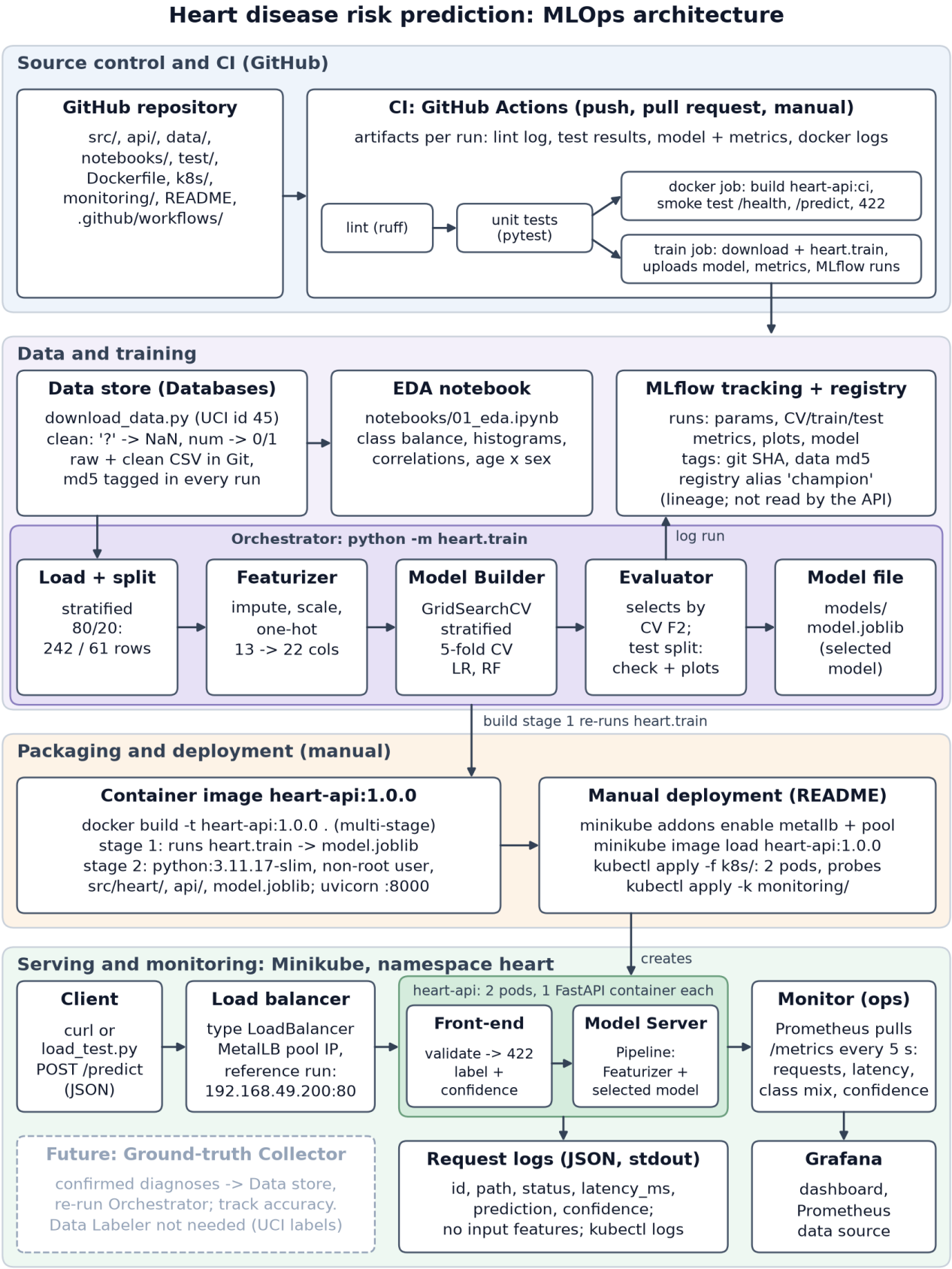


Figure 1: End-to-end architecture. Box titles use the components of a supervised-learning system from CS-1.3 (p6): Orchestrator, Featurizer, Model Builder, Evaluator, Model Server, Front-end, Performance Monitor, Ground-truth Collector, Data Labeler and Databases.

The training workflow is a script, `python -m heart.train`, which the course allows: the Orchestrator's "workflow might be implemented as a simple script" (CS-1.3, p13). GitHub Actions triggers it on every push. Serving follows the online (synchronous) pattern "client → load balancer → model server (replicas)" (CS-7, p28).

3. Data acquisition and exploratory analysis

`scripts/download_data.py` downloads the raw file from UCI (with a committed copy as fallback) and writes the cleaned table `data/processed/heart_clean.csv`. Cleaning follows the lifecycle step "clean (replace, impute, remove outliers and duplicates)" (CS-1.2, p16):

- The raw file uses `?` for unknown values. They are read as missing: 6 of 303 rows have one (4 in `ca`, 2 in `thal`). They are not filled in the CSV; the model pipeline imputes them (median for numeric columns, most frequent value for categorical ones) with statistics from the training split only, so no test information leaks into training.
- Duplicate rows: 0. Every value must parse as a number and `num` must be 0 to 4, or cleaning stops with an error.
- Outliers: the extreme values visible in the histograms (for example the highest cholesterol and ST-depression readings) are clinically plausible measurements, not entry errors, so they are kept; removing them would drop real patients from a small dataset.
- Encoding is part of the model pipeline (Section 4), so the same transformation runs in training and in the API.

Class balance. 164 patients without and 139 with disease (minority/majority ratio 0.85). The classes are close to balanced, so accuracy stays meaningful, and every split and fold is stratified to keep this ratio.

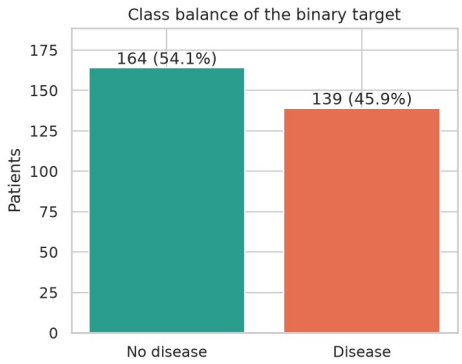


Figure 2: Class balance of the binary target.

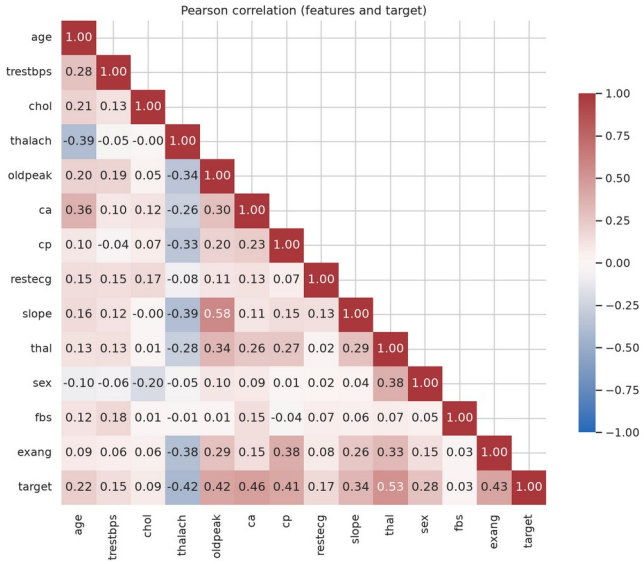


Figure 3: Pearson correlation of features and target.

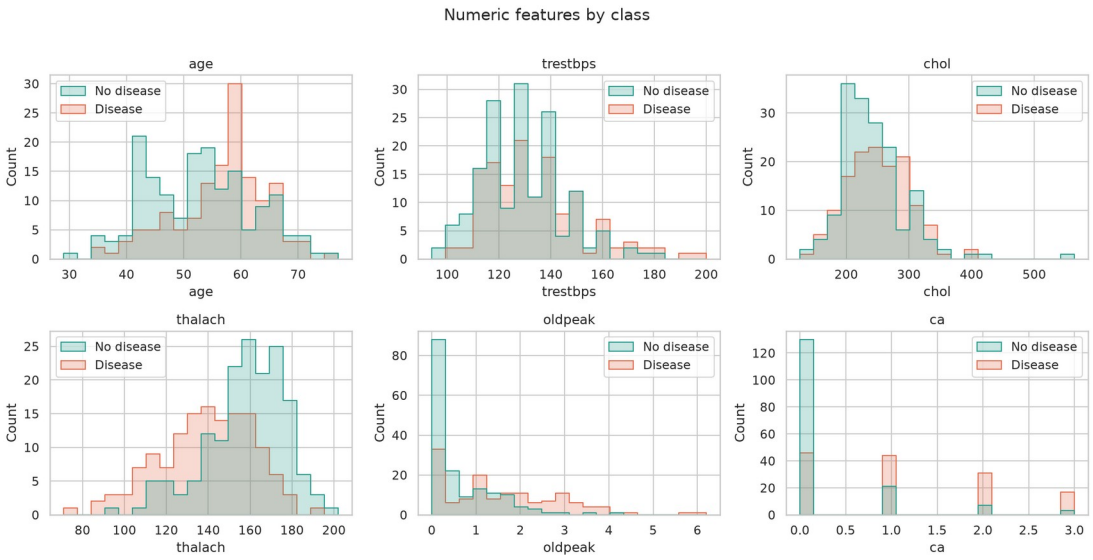


Figure 4: Distributions of the six numeric features by class.

Observations. The numeric features that separate the classes most, in standard-deviation units, are `ca` (0.92), `oldpeak` (0.85) and `thalach` (0.84). The strongest correlations with the target are `thal` (+0.53), `ca` (+0.46), `exang` (+0.43) and `oldpeak` (+0.42). The strongest pair of features, `oldpeak-slope` (+0.58), is not strong enough to make either redundant, so all 13 features are kept. The disease rate is 55.3% for men and 25.8% for women, but only 97 of the 303 patients are women, so per-sex estimates rest on small groups. The notebook `01_eda.ipynb` also shows the disease rate per category and per age band and sex.

4. Feature engineering and model development

Features. A scikit-learn `ColumnTransformer` prepares the 13 features: the six numeric ones (age, trestbps, chol, thalach, oldpeak, ca) get median imputation and standard scaling; the four categorical codes without numeric order (cp, restecg, slope, thal) get most-frequent imputation and one-hot encoding; the three binary ones (sex, fbs, exang) are kept as 0/1. The result has 22 columns. `ca` (0–3 vessels) is ordered, so it stays numeric.

Models and tuning. Two candidates, each a `Pipeline` of the preprocessing plus a classifier: logistic regression, the simple and interpretable baseline, and a random forest, to test whether non-linear interactions add signal. The data is split once, stratified, into 242 training and 61 test rows (seed 42). On the training split, `GridSearchCV` with stratified 5-fold cross-validation tunes each model (16 combinations of C, L1/L2 penalty and class weighting for logistic regression; 24 of trees, depth, leaf size and class weighting for the forest) and refits on **F2**, the same recall-weighted metric that then chooses between the two models: the candidate with the highest **cross-validated F2** is selected, with ROC-AUC breaking ties. The test split is used only for the final check.

Cross-validation (mean ± std, 5 folds)	Accuracy	Precision	Recall	F1	F2	ROC-AUC
Logistic regression	0.855 ± 0.030	0.861 ± 0.075	0.828 ± 0.054	0.840 ± 0.030	**0.832 ± 0.038**	0.907 ± 0.019
Random forest	0.826 ± 0.037	0.830 ± 0.059	0.792 ± 0.095	0.805 ± 0.046	0.796 ± 0.074	0.895 ± 0.027

Held-out test split (61 rows)	Accuracy	Precision	Recall	F1	F2	ROC-AUC
Logistic regression (selected)	0.885	0.818	0.964	0.885	0.931	0.959
Random forest	0.885	0.862	0.893	0.877	0.887	0.945

Selection. Logistic regression wins on CV F2 (0.832 vs 0.796) with C = 1.0, L2 penalty and `balanced` class weights; the forest's best setting was 400 trees, depth 3, leaf size 1. On the test split the selected model finds 27 of the 28 disease cases (1 missed) at the cost of 6 false alarms, which is the trade-off the problem asks for. Its test ROC-AUC (0.959) is above its training ROC-AUC (0.929): with 61 test rows one patient moves recall by 0.036, so the test figures are a check, not a precise estimate; the cross-validated means and their spread are the better guide.

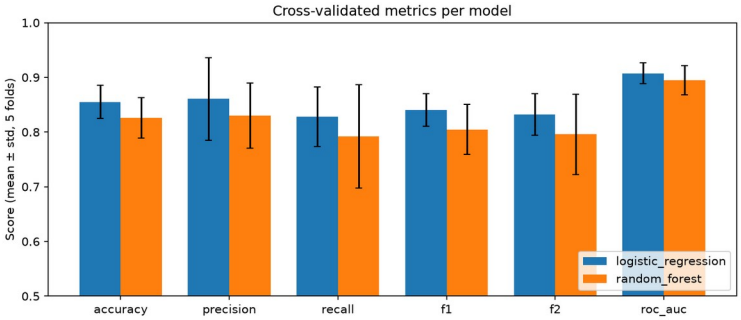


Figure 5: Cross-validated metrics per model.

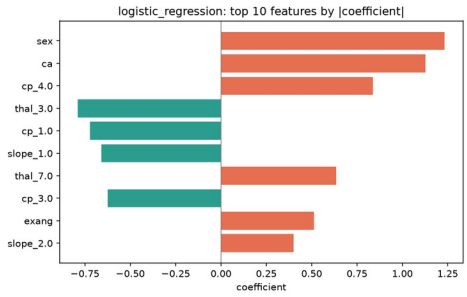


Figure 6: Selected model: largest coefficients.

Largest coefficients of the selected model: `sex` (+1.23), `ca` (+1.13), `cp_4.0` (+0.84) and `thal_3.0` (-0.79). Sex carries the largest weight. The course defines bias as "imbalances in the accuracy of predictions across different groups, such as age" (CS-1.2, p17); with 97 women in the data, recall per sex should be checked before any clinical use (Section 10).

5. Experiment tracking with MLflow

MLflow covers the three parts of a tracking system named in CS-4 (p8): the **experiment database** (SQLite `mlflow.db` plus the `mlruns/` artifact store), the **dashboard** (`mlflow ui`) and the **client library** (the `mlflow` calls in `src/heart/train.py`). Each training run logs one MLflow run per candidate, following the course's list of what to track (CS-4, p16 and p36):

- **Parameters:** the best hyperparameters, grid size, CV folds, refit metric, split size, seed and the preprocessing settings (imputers, scaler, encoder).
- **Metrics:** accuracy, precision, recall, F1, F2 and ROC-AUC, as CV mean and std, on the training split and on the test split.
- **Tags:** git commit SHA, MD5 of the raw and clean data files and of `requirements.txt`, so each run records which code, data and environment produced it.
- **Artifacts and plots:** confusion matrix, ROC and precision-recall curves on the test split; the full grid-search table, test-split predictions and feature importances; `requirements.txt` and the `Dockerfile`; a data card; and the model with its signature and an input example.

Each training creates one parent run per model and, under it, one child run per grid combination (16 for logistic regression, 24 for the forest) with that combination's parameters, its cross-validated scores and its fold-by-fold results (`cv_folds.json`); the child runs are the tuning trials. The selected model is registered as `heart-disease-classifier`, version 2, with the alias `champion`. The experiment holds 84 runs from the repository's two trainings (the command line and the training notebook); `reports/mlflow_runs.csv` exports the runs table.

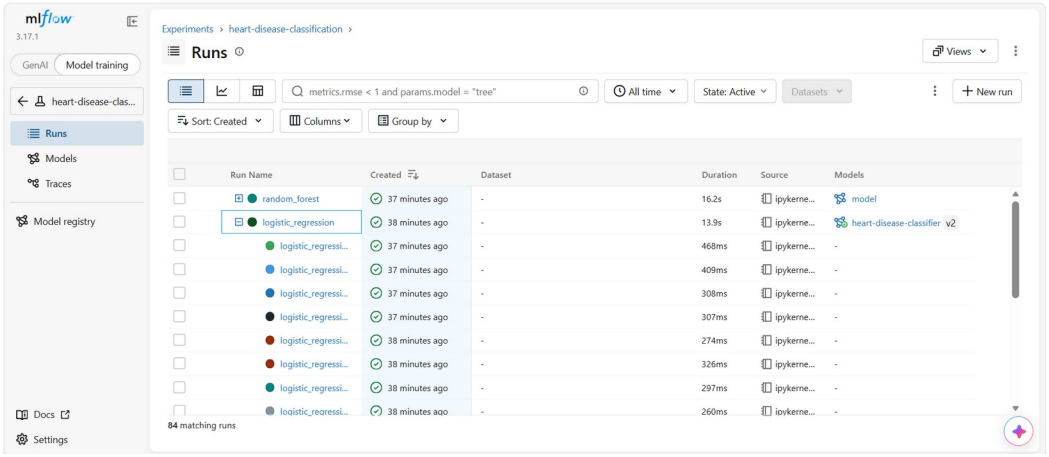
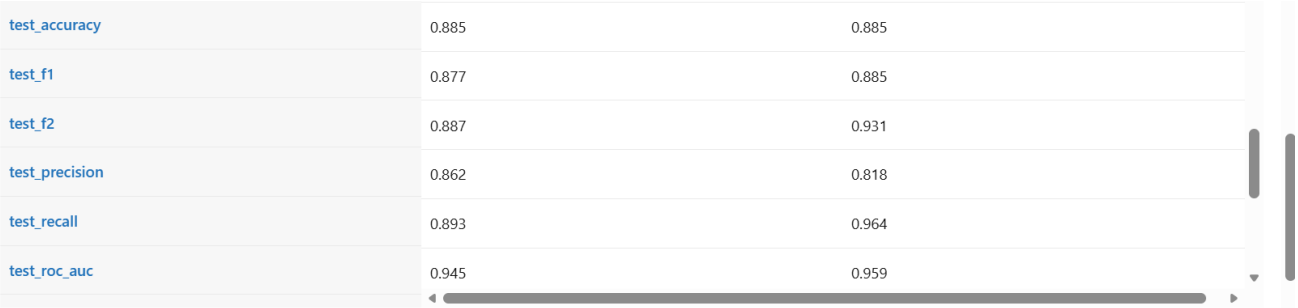


Figure 7: MLflow: parent runs per model, one expanded to its grid-search child runs; the selected run links to the registered model.



test_accuracy	0.885	0.885
test_f1	0.877	0.885
test_f2	0.887	0.931
test_precision	0.862	0.818
test_recall	0.893	0.964
test_roc_auc	0.945	0.959

Figure 8: MLflow comparison of the two final runs on the test split (left: random forest, right: logistic regression).

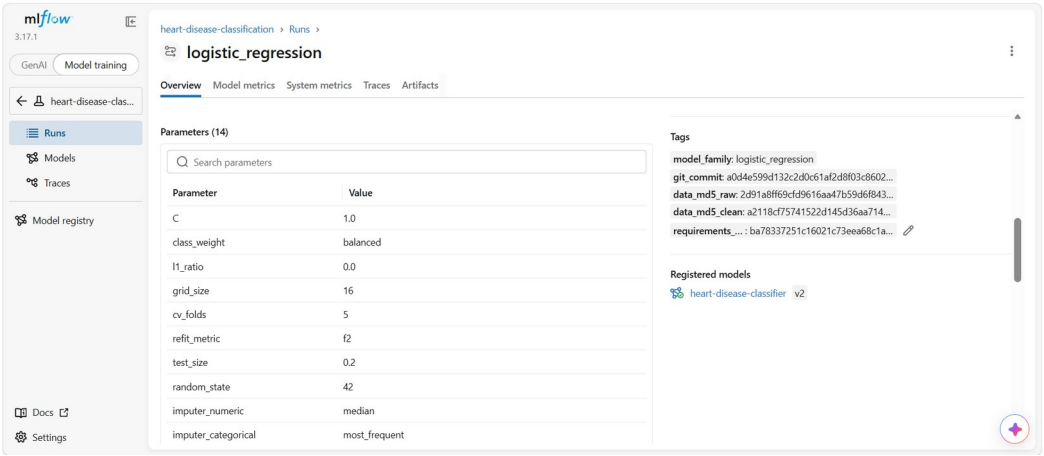


Figure 9: The selected run: parameters, and tags with the git commit and data checksums.

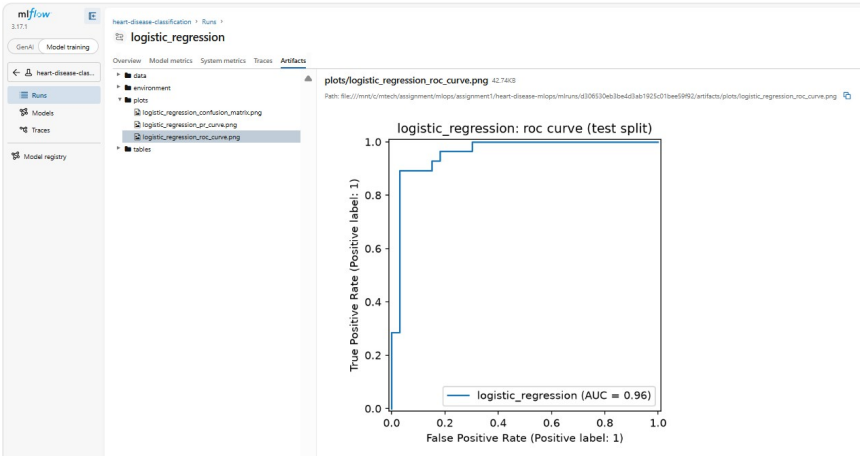


Figure 10: The selected run's artifacts: data card, environment files, plots and tables, here the ROC curve.

6. Model packaging and reproducibility

The final model is saved in two reusable formats. `models/model.joblib` is the fitted `Pipeline`, preprocessing and classifier in one object, serialised with joblib, scikit-learn's pickle-based format ("saves models as pickled python objects", CS-5, p10); this is the file the API loads. The same model is logged to MLflow with an `MLmodel` YAML descriptor ("mlflow stores a yaml file", CS-5, p9), serialised with skops, which only loads the types it is told to trust. ONNX, the framework-independent format (CS-5, p8), was not used: "portable models behave in subtly different ways than the original" (CS-6, p9), and a pipeline with imputation and one-hot encoding is exactly where that risk lies.

Because preprocessing lives inside the pipeline, the API sends raw patient fields to the same fitted imputers, scaler and encoder that training used. Reproducibility rests on a pinned `requirements.txt` (generated from a short `requirements.in`), a smaller `requirements-serve.txt` for the image, a fixed seed (42) for the split, the folds and the forest, and the lineage tags above. Retraining reproduces every metric, and the model trained inside the Docker image gives the same predictions as the committed model on all 303 rows, which CI checks on every push (`scripts/check_model_parity.py`). On the same hardware the file is also byte-identical; across CPUs the bytes can differ, because the numerical libraries pick CPU-specific routines, so reproducibility is judged on predictions. The API reports the file's MD5 prefix as its version, `1.0.0+5b799e17`.

7. CI/CD pipeline and automated testing

Unit tests. `test/` holds 41 pytest test cases: data processing (8: raw shape, ? read as missing values, binary target, rejection of bad input, round trip, summary counts), features (4: no missing values after imputation, one-hot expansion, scaling, unseen categories), models (8: grids, training and probabilities, beating the majority class, reproducibility, the prediction parity check), training (4: complete metrics, sorted feature importances, MLflow parent and child runs with their artifacts) and the API (17: health, prediction and confidence, unknown values, invalid and missing fields rejected with 422, metrics and their labels, health probes not counted as traffic, an internal error returned as JSON with a request id). They run in 16.1 s, following "test entire system, not just the model" (CS-1, p74).

GitHub Actions (`.github/workflows/ci.yml`) runs on every push and pull request, and manually. Jobs: **lint** (ruff check and format check) → **unit tests** (pytest with a JUnit report) → in parallel, **train** (download, `python -m heart.train`, score the sample patient, write the metrics to the run summary) and **docker** (build the image, smoke-test `/health`, `/predict` and a 422 for invalid input, and check that the model trained inside the image predicts the same as the committed one). Every job uploads its log or results as an artifact, even when it fails: `lint-log`, `test-results`, `model-and-metrics` (model, metrics, MLflow runs) and `docker-logs`. Each step runs with `bash -eo pipefail`, so a failing command fails the job even when its output is piped to a log, and the job stops with that step's log. A failing test on a separate branch shows the pipeline stopping at the test job (below).

Of the course's Continuous X (CS-1, p71), the pipeline covers continuous integration and continuous training: every push re-runs the tests and retrains. Deployment to the cluster is a manual step (Section 9).

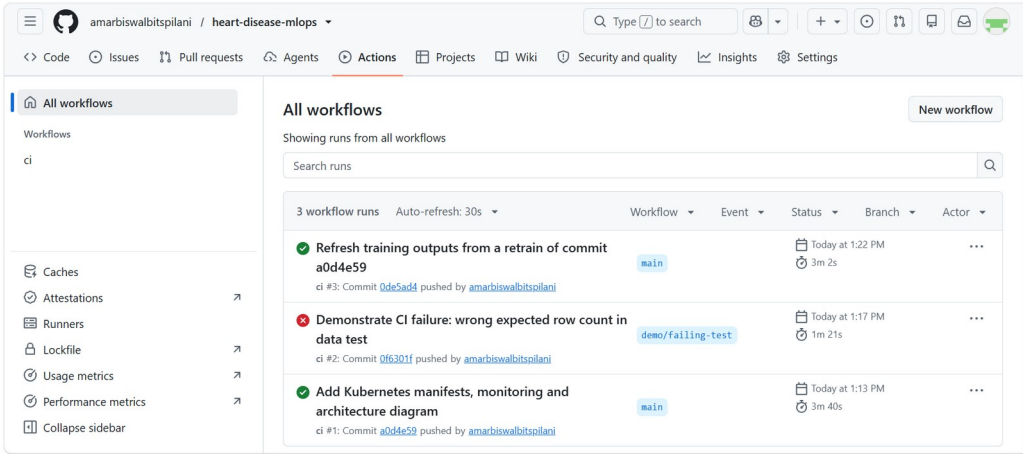


Figure 11: GitHub Actions: green runs on main and the red run of the failing-test branch.

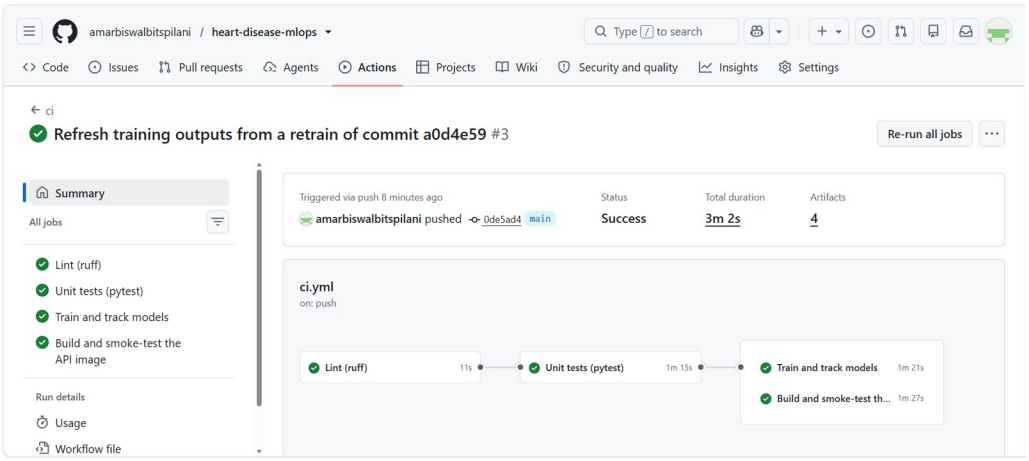


Figure 12: A green run: lint, then unit tests, then training and the Docker job in parallel.

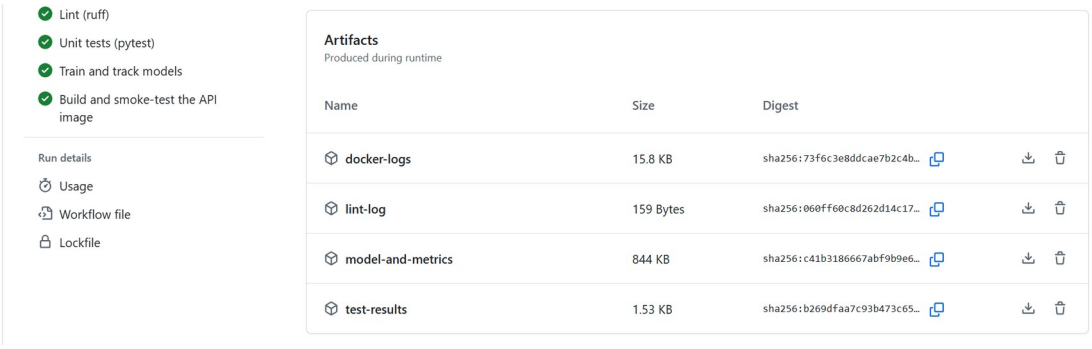


Figure 13: The same run's uploaded artifacts.

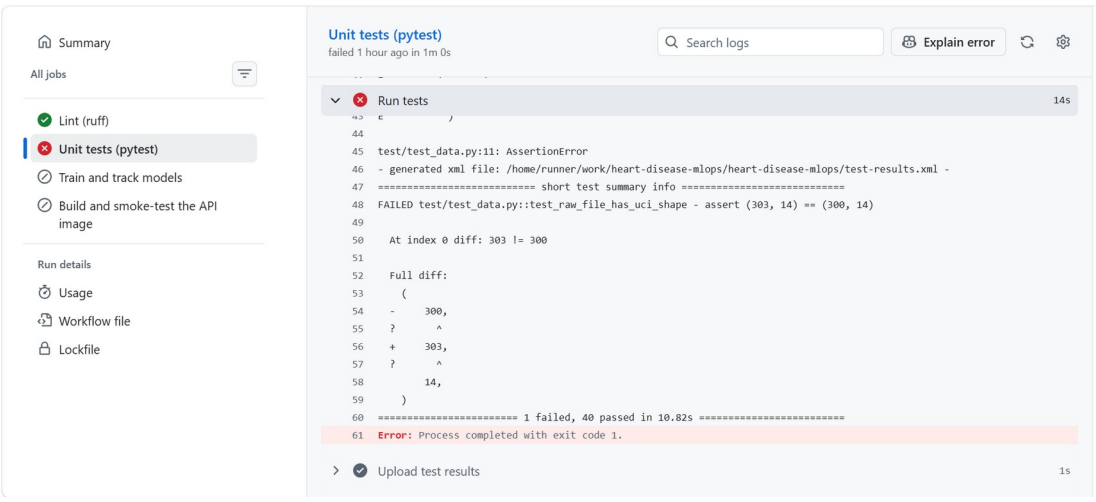


Figure 14: The failing run: the test step stops with the assertion in the log, the training and Docker jobs are skipped, and the test results are still uploaded.

8. Model containerisation

The API is FastAPI (`api/app.py`). `POST /predict` accepts a JSON patient record with readable field names (for example `chest_pain_type`, `max_heart_rate`, `major_vessels`); Pydantic bounds and allowed codes reject impossible values with HTTP 422 before the model is called, and `major_vessels` and `thalassemia` may be `null`, which the pipeline imputes. The response holds the prediction, its label, the probability of disease, the **confidence** (probability of the predicted class) and the model version. For the repository's sample patient the container returned the following (the Kubernetes deployment returns the same, Section 9):

```
{"prediction": 1, "label": "disease", "probability_of_disease": 0.5779, "confidence": 0.5779,
 "model_version": "1.0.0+5b799e17"}
```

`GET /health` serves the Docker and Kubernetes probes, and `GET /metrics` exposes Prometheus metrics.

The `Dockerfile` is multi-stage. Stage 1 (`python:3.11.17-slim`) installs the training requirements, copies the code and the committed data, and runs `python -m heart.train`, so the image always carries a model trained from the code it ships with. Stage 2 starts again from `python:3.11.17-slim`, installs only `requirements-serve.txt`, copies `src/heart/`, `api/` and `models/model.joblib` from stage 1, runs as a non-root user and starts uvicorn on port 8000 with a `/health` health check. This matches the course's training-only and inference containers (CS-5, p32 and p37) and its advice to "configure /health or /ready endpoints" (CS-5, p51). The image `heart-api:1.0.0` is 156 MB (content size; 685 MB unpacked on disk) and runs Python 3.11.17.

```
+ docker build --network=host -q -t heart-api:1.0.0 .
sha256:dcab93778c9bc7050921e7065205120491cfa82163abae043e5577d818029c55
+ docker run -d --name heart-api -p 8000:8000 heart-api:1.0.0
bf62d5db542eb46009e9e467c0bee3da290f90e067d01df0cb68091c3cfeb585
+ sleep 5
+ curl -s localhost:8000/health
{"status":"ok","model_version":"1.0.0+5b799e17"}
+ curl -s -X POST localhost:8000/predict -H 'Content-Type: application/json' -d @api/sample_input.json
{"prediction":1,"label":"disease","probability_of_disease":0.5779,"confidence":0.5779,"model_version":"1.0.0+5b799e17"}
```

Figure 15: Building the image, running the container, and calling /health and /predict with the sample input.

9. Production deployment on Kubernetes

The image runs on Minikube (Docker driver). The manifests in `k8s/` create a namespace, a Deployment with **2 replicas** (readiness and liveness probes on `/health`, CPU and memory requests, a memory limit, and a locked-down container: non-root user, read-only root filesystem, no privilege escalation, all Linux capabilities dropped, default seccomp profile) and a **Service of type LoadBalancer**. Minikube's MetalLB addon assigns the Service an external IP from a pool inside Minikube's own network, so the same manifest gets a cloud load balancer on GKE, EKS or AKS, once the image is pushed to a registry the cluster can pull from. In the course's terms this is a model-as-a-service ("live-scoring model", CS-6, p8) deployed as a dynamic deployment on a server (CS-6, p22) inside a container (CS-6, p25), served online and synchronously behind a load balancer (CS-7, p28). Batch scoring was not chosen because predictions are needed per patient, on demand, and batch serving requires that "the input feature space must be finite" (CS-7, p20).

Verification: both API pods were ready (2/2), and `/health` and `/predict` answered through the EXTERNAL-IP `192.168.49.200` (Section 8 shows the response).

```
+ kubectl -n heart get deployment heart-api -o wide
NAME        READY   UP-TO-DATE   AVAILABLE   AGE     CONTAINERS   IMAGES            SELECTOR
heart-api   2/2     2            2           2d18h   api          heart-api:1.0.0   app=heart-api
+ kubectl -n heart get pods,service -l app=heart-api
NAME                             READY   STATUS    RESTARTS      AGE
pod/heart-api-5588c45778-p6m4r   1/1     Running   1 (80m ago)   13h
pod/heart-api-5588c45778-pfwft   1/1     Running   1 (80m ago)   13h

NAME                 TYPE           CLUSTER-IP       EXTERNAL-IP      PORT(S)        AGE
service/heart-api    LoadBalancer   10.108.220.242   192.168.49.200   80:30914/TCP   2d18h
++ kubectl -n heart get service heart-api -o 'jsonpath={.status.loadBalancer.ingress[0].ip}'
+ IP=192.168.49.200
+ curl -s http://192.168.49.200/health
{"status":"ok","model_version":"1.0.0+5b799e17"}
+ curl -s -X POST http://192.168.49.200/predict -H 'Content-Type: application/json' -d @api/sample_input.json
{"prediction":1,"label":"disease","probability_of_disease":0.5779,"confidence":0.5779,"model_version":"1.0.0+5b799e17"}
```

Figure 16: kubectl: the Deployment (2/2), its pods and the LoadBalancer Service with its EXTERNAL-IP; /health and /predict called through it.

Release strategy. Updates use the Deployment's rolling update, which replaces pods gradually. The course lists blue/green, canary, A/B and shadow deployment as ways to "reduce downtime and risks" (CS-1.2, p26), and warns that "offline evaluations alone are not sufficient" (CS-9, p37). The next step is a canary: a second Deployment with the new model behind the same Service, taking a small share of traffic; the prediction counter already carries a `model_version` label to compare the two.

10. Monitoring and logging

Request logging. Every `/predict` request writes one JSON line to stdout (`kubectl logs`): request id, method, path, status, latency in ms, prediction and confidence. Patient feature values are never logged; health data falls under regulations "such as GDPR and HIPAA" (CS-7, p61), and the request id is enough to trace a call.

Metrics and dashboard. Prometheus scrapes `/metrics` from each pod every 5 s, finding the pods through a headless Service (2 of 2 targets up). The metrics are request counts by endpoint and status, a latency histogram, predictions by class (labelled with the model version) and a confidence histogram. A Grafana dashboard, provisioned from the repository, shows replicas up, predictions, error ratio, p95 latency, request rate, latency over time, predictions by class, requests per pod and the confidence distribution. `scripts/load_test.py` sent 300 requests through the LoadBalancer in 16.1 s, each with a 5% chance of an invalid age: 283 returned 200 and 17 returned 422, and Prometheus counted the same 283 and 17. The p95 latency of `/predict` in that run was about 22.0 ms (interpolated within the 10–25 ms histogram bucket), well inside the course's guideline of "latency sla < 200ms" for online serving (CS-7, p28).



Figure 17: Load test through the LoadBalancer, then the JSON request logs of both pods (`kubectl logs`).

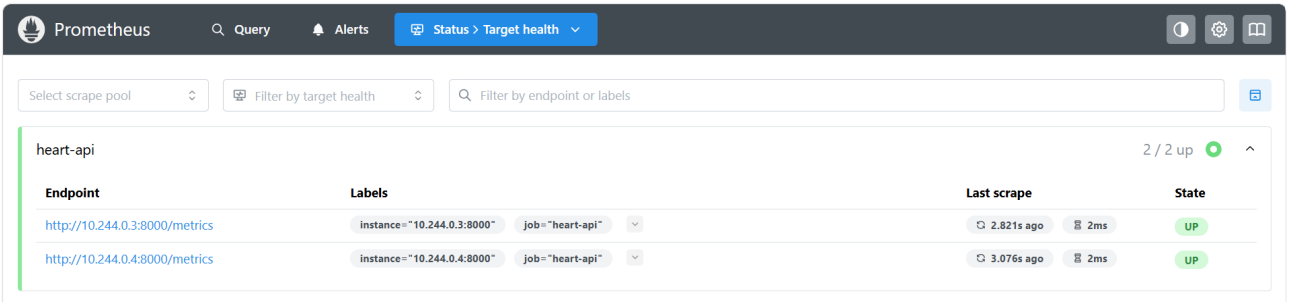


Figure 18: Prometheus: both API pods scraped and up.



Figure 19: Grafana after two load tests: replicas up, predictions, error ratio (the 5% invalid requests), p95 latency, request rate and latency. Red and green are Grafana's default colour thresholds, not alerts.



Figure 20: Grafana, continued: predictions per minute by class, requests per pod (both replicas share the traffic) and the confidence distribution.

Scope. This monitors the system: "lag and throughput" (CS-1.3, p9), errors, and the class mix and confidence of predictions, an early signal of a shift in the inputs. Data drift and concept drift (CS-1.2, p28) and live accuracy need confirmed diagnoses, which a Ground-truth Collector would provide; they are future work.

Maturity and next steps. Training is automated on every push, but deployment is manual: Azure's level 2, "automated training", not yet level 3, "automated model deployment" (CS-2, p20), and Stage 1 of continual learning, "manual, stateless retraining" (CS-9, p29). The tools were chosen on the course's 4Cs, cost, coverage, complexity and community (CS-1, p49): all are free for this use (GitHub Actions for public repositories, the rest open source), and MLflow, GitHub Actions, Docker, Minikube, Prometheus and Grafana cover tracking, CI, packaging, serving and monitoring without a platform team. Limitations: 303 patients from one site (the Cleveland database) and a 61-row test split; sex as the strongest feature, unchecked for per-group recall; the default decision threshold of 0.5, which could be lowered to trade precision for even higher recall; a local cluster. Next: per-sex recall, a canary release, serving straight from the registry's `champion` (the course's deployment phase takes "artifacts … from the model registry", CS-1.2, p25), and drift alerts.

References

1. A. Janosi, W. Steinbrunn, M. Pfisterer, R. Detrano. *Heart Disease* [dataset]. UCI Machine Learning Repository, 1989. https://doi.org/10.24432/C52P4X
2. Course lectures, Machine Learning Operations (AIMLCZG523), BITS Pilani WILP: CS-1 Introduction to MLOps; CS-1.2 Machine Learning Life Cycle; CS-1.3 Machine Learning Architecture; CS-2 Motivation and Drivers for MLOps; CS-4 Model Experimentation; CS-5 Model File Formats and Serialization; CS-6 Deployment Patterns and Orchestration; CS-7 Batch and Online Prediction; CS-9 Test in Production.
3. Documentation: scikit-learn, MLflow, FastAPI, Pydantic, pytest, Ruff, GitHub Actions, Docker, Kubernetes, Minikube, MetalLB, Prometheus, Grafana.